STUDENT PROJECT GUIDE

HireFlow AI

Build an Autonomous AI Recruitment Platform — Step by Step

7 AI Agents you will build:

1. JD Parser 2. Resume Screener 3. Question Generator
4. Email Sender 5. Answer Evaluator 6. AI Interviewer 7. Scheduler

Tech Stack: Python · Flask · React · MongoDB · LangGraph · Gemini · Resend

Table of Contents

1. Introduction — What Are You Building? 3
2. Why This Project Matters (Learning Outcomes) 3
3. System Architecture Overview 4
4. The Complete Agent Pipeline 4
5. Tech Stack & Why Each Tool Was Chosen 6
6. Project Roadmap — Phase by Phase 6
7. Database Design (MongoDB Collections) 9
8. Deep Dive — Each of the 7 Agents 10
9. Authentication & Multi-Tenancy Explained 12
10. AI Prompt Engineering Recommendations 12
11. Common Errors & How to Debug Them 14
12. Evaluation Checklist — Is My Project Complete? 14
13. Stretch Goals (For Extra Marks) 15

1. Introduction — What Are You Building?

HireFlow AI is a real, production-style SaaS platform that automates the recruitment process for a company's Talent Acquisition (TA) team. Instead of recruiters manually reading every resume and writing every interview question by hand, HireFlow AI uses a pipeline of **7 specialized AI agents** — each one responsible for exactly one job — to take a hiring process from "we have a job opening" all the way to "here is our final hiring decision."

Think of it like an assembly line in a factory. Each agent is a station on the line. A job description goes in at one end, and a fully evaluated, scheduled candidate comes out the other end. No single agent tries to do everything — and that is the most important engineering lesson of this entire project.

> **WHY "AGENTS" AND NOT JUST "FUNCTIONS"?**
> An AI agent is a unit of the system that uses a Large Language Model (LLM) to make a judgment call — not just run fixed logic. A normal function might check "is score > 50". An agent reads a resume and a JD in natural language and decides "this candidate is a strong match because of X, Y, Z" — that reasoning step is what makes it an agent rather than a script.

By the end of this project you will have built a multi-tenant SaaS application — meaning multiple companies can sign up and use your platform independently, each with their own isolated data — with a secure login system, a working AI pipeline, and a polished React dashboard. This is portfolio-grade work; treat it as such.

2. Why This Project Matters (Learning Outcomes)

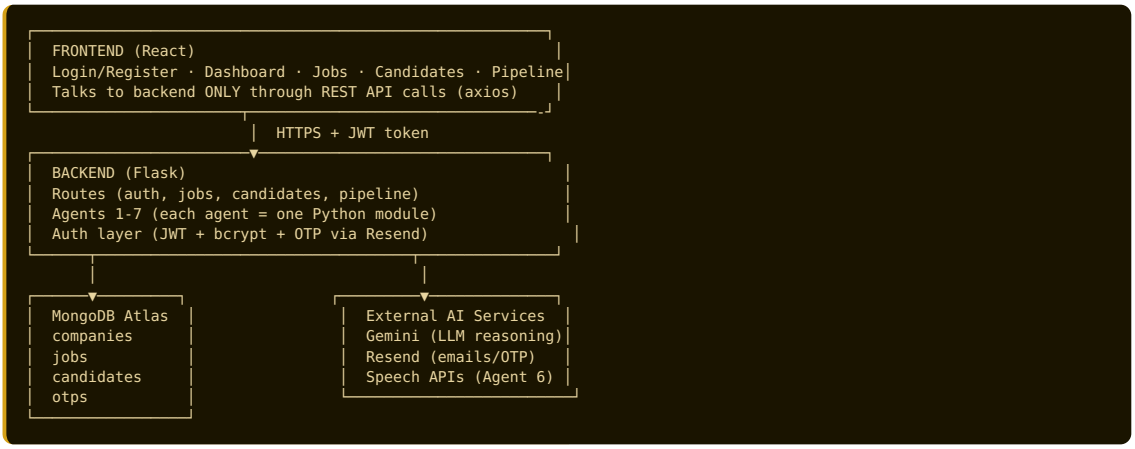
This project is intentionally designed to touch almost every skill a modern full-stack / AI engineer needs. Here is exactly what you will practice, and why each skill matters in the real world:

Skill	Where you practice it	Why it matters
REST API design	Flask routes for jobs, candidates, pipeline	Every backend job in the industry requires this
NoSQL database design	MongoDB collections with multi-tenancy	Most modern SaaS products use document databases
LLM prompt engineering	All 7 agent prompts	The single most in-demand AI skill right now
Agent orchestration	Chaining Agent 1 → 2 → 3 → ... → 7	This is what "AI engineering" means beyond just calling an API
Authentication & security	JWT, bcrypt password hashing, OTP verification	Every real product needs secure user accounts
Multi-tenant architecture	company_id scoping on every collection	This is how every B2B SaaS company (Slack, Notion, etc.) is built internally
React frontend development	Dashboard, modals, state management	Translates AI output into something a human can actually use
Third-party API integration	Gemini, Resend (email), and later speech APIs	Real products are glue between many specialized services
Error handling at scale	Global Flask error handlers	Separates a "demo" from a "product"

> **HOW TO THINK ABOUT THIS PROJECT**
> Don't think of this as "build 7 agents." Think of it as "design one clean, repeatable pattern for an agent, then apply that pattern 7 times." Agent 1 and Agent 5 will feel structurally identical even though they do completely different things — same prompt template structure, same error handling, same JSON-in-JSON-out contract. Once you see that pattern, the project gets dramatically easier.

3. System Architecture Overview

Before touching any code, you need a mental map of how the pieces connect. HireFlow AI has three layers, and understanding the boundary between them is critical:

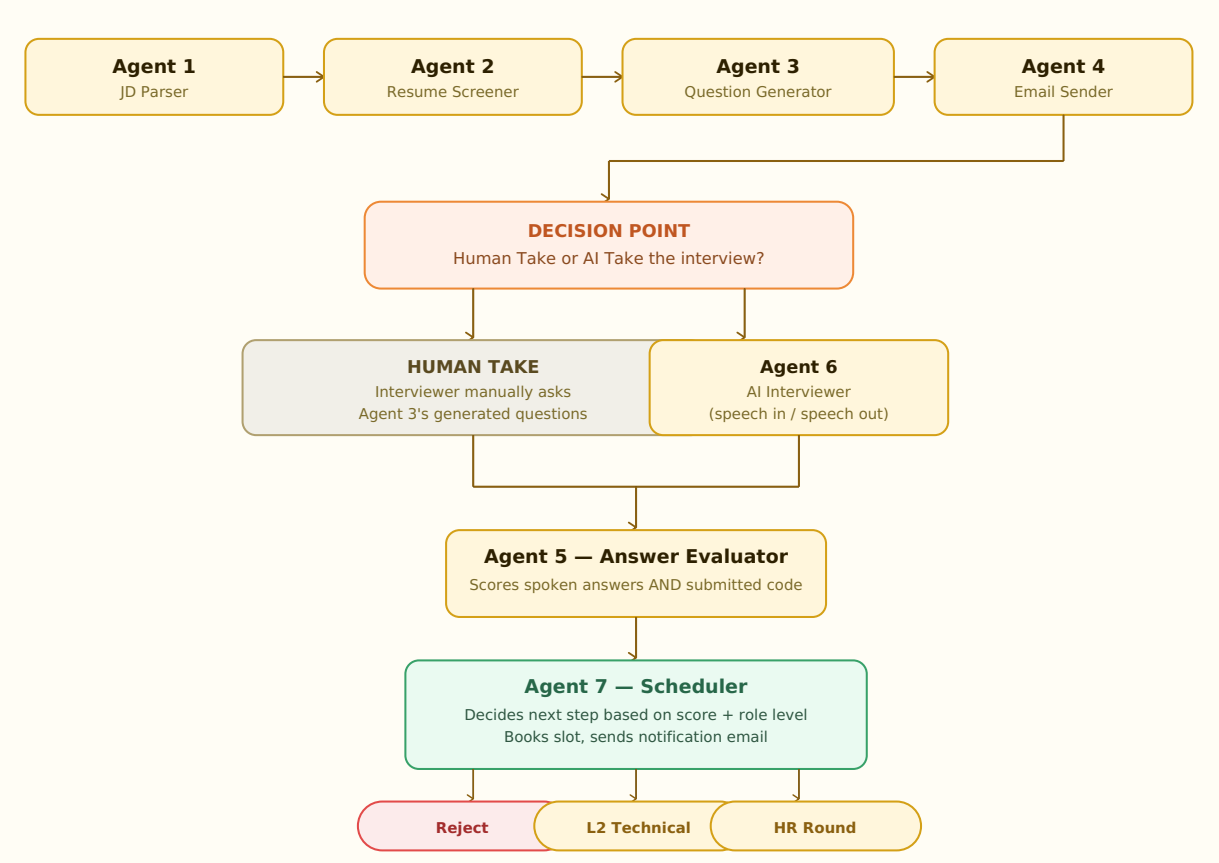


The golden rule of this architecture: **the frontend never talks to MongoDB or Gemini directly.** Everything flows through Flask. This is not arbitrary — it means your API keys and database credentials stay on the server, never exposed in browser code where anyone could steal them.

> **WHY FLASK IN THE MIDDLE?**
> If React called Gemini directly, your Gemini API key would be sitting in your frontend JavaScript bundle — visible to literally anyone who opens browser dev tools. Flask acts as a secure middleman: the browser asks Flask, Flask asks Gemini using a key only Flask knows, Flask sends back just the result.

4. The Complete Agent Pipeline

This is the heart of the project. Study this diagram until you can redraw it from memory — every agent you build fits into exactly one of these boxes.



#	Agent	One-line job	Input	Output
1	JD Parser	Turns raw JD text into structured data	JD text / PDF / Word	JSON: title, skills, experience, responsibilities
2	Resume Screener	Scores a resume against a JD	Resume + JD data	JSON: match score, matched/missing skills, recommendation
3	Question Generator	Writes personalized interview questions	Candidate profile + JD	JSON: 8 categorized questions
4	Email Sender	Writes & sends the shortlist email	Candidate profile + JD	Email delivered via Resend
5	Answer Evaluator	Scores interview answers (spoken or code)	Transcribed answers / code submission	JSON: evaluation + hiring recommendation
6	AI Interviewer	Conducts the interview by voice	Agent 3's questions	Transcribed text answers (speech-to-text)
7	Scheduler	Decides & books the next round	Agent 5's recommendation + role level	Calendar booking + notification email

IMPORTANT SCOPE NOTE FOR STUDENTS

Agents 1-5 are fully achievable with the tools in this guide (Gemini + Flask + MongoDB). Agents 6 and 7 require additional services — Agent 6 needs a speech-to-text/text-to-speech API (e.g. browser Web Speech API for a simpler version, or a cloud STT/TTS service for a robust one), and Agent 7 needs calendar integration (e.g. a simple internal scheduling table is acceptable for a college project — full Google Calendar integration is a stretch goal, not a requirement). Build 1-5 first and get them rock solid before attempting 6 and 7.

5. Tech Stack & Why Each Tool Was Chosen

Every tool in this stack was picked for a reason. Understanding *why* matters more than memorizing the list — in your viva/interview, you should be able to defend every choice.

Backend: Python + Flask

Flask is a lightweight web framework — it doesn't force a rigid structure on you, so it's easier to learn than heavier frameworks like Django. You write a function, decorate it with a URL, and it becomes an API endpoint. This simplicity lets you focus on the AI logic rather than fighting the framework.

Database: MongoDB

MongoDB stores data as JSON-like documents instead of rigid rows and columns. A candidate's data — name, skills, scores, interview questions, evaluation — naturally nests together as one document. In a traditional SQL database, this same data would be split across 4-5 tables joined together, which is unnecessary complexity for this use case.

AI Orchestration: LangChain / LangGraph

LangChain gives you reusable building blocks — `PromptTemplate`, chat model wrappers — so you don't write raw HTTP requests to the AI provider yourself. LangGraph (used when you connect agents together) lets you define your pipeline as a graph of steps with conditional branches — exactly like the Human Take / AI Take decision point in your pipeline.

LLM Provider: Google Gemini

Gemini offers a genuinely usable free tier for student projects, and the `langchain-google-genai` package integrates it directly into LangChain's interface. Any of Gemini's chat models can be swapped in with one line of config — this flexibility matters because AI model pricing and availability changes often.

Frontend: React + Vite

React's component model maps naturally onto this UI: a Sidebar component, a JobCard component, a CandidateModal component — each independent and reusable. Vite (the build tool) gives near-instant reload during development, which matters a lot when you're iterating quickly on UI.

Authentication: JWT + bcrypt

bcrypt hashes passwords so even if your database were leaked, raw passwords are never exposed. JWT (JSON Web Token) lets the backend verify "who is this request from" without a database lookup on every single request — the token itself carries the company's identity, signed so it can't be forged.

Email/OTP: Resend

Resend is a modern email API with a generous free tier and a clean Python SDK — a few lines of code sends a fully formatted HTML email. It's used for both OTP verification emails (security) and shortlist notification emails (Agent 4).

6. Project Roadmap — Phase by Phase

Follow this exact order. Each phase produces something runnable — never move to the next phase until the current one works end-to-end. This is how real engineering teams build software, and it prevents the classic student mistake of writing 2000 lines of code before testing anything.

PHASE 0 — Setup (Day 1)

- Install Python 3.10+, Node.js 18+, Git, and a code editor (VS Code recommended)
- Create a MongoDB Atlas account (free tier) and get your connection string
- Create a Google AI Studio account and get a Gemini API key
- Create a Resend account and verify a sending domain (or use their test domain)

PHASE 1 — Foundation (Days 2-3)

- Create the monorepo structure: `backend/` and `frontend/` folders
- Set up a Python virtual environment, install Flask + pymongo + python-dotenv
- Build the Flask application factory pattern (`create_app()`) with blueprints
- Confirm MongoDB connects, confirm Gemini responds to a basic test prompt
- Scaffold React with Vite, confirm it runs on localhost
- **Checkpoint:** `/api/health` returns 200, React shows the default page

PHASE 2 — Agent 1: JD Parser (Days 4-5)

- Write the prompt template that extracts structured fields from raw JD text
- Build the `parse_jd()` function — call Gemini, parse JSON response, validate it
- Build the file extractor utility (PDF and Word text extraction)
- Build the `/api/jobs/` POST route that ties it all together and saves to MongoDB
- **Checkpoint:** POST a JD, get back clean structured JSON, see it in MongoDB Atlas

PHASE 3 — Agent 2: Resume Screener (Days 6-7)

- Write the screening prompt — compares resume text against the JD's structured data
- Build `screen_resume()` — same pattern as Agent 1, different prompt and output shape
- Build the `/api/candidates/job/<job_id>/screen` route
- **Checkpoint:** upload a resume, get back a match score with matched/missing skills

PHASE 4 — Agents 3 & 4: Questions + Email (Days 8-10)

- Build the question generator — note it uses **higher temperature** (more creative) than Agents 1-2
- Build the email content generator + Resend integration for actually sending it
- Build the pipeline routes that chain these onto an already-screened candidate
- **Checkpoint:** generate questions for a candidate, then receive a real shortlist email in your inbox

PHASE 5 — Authentication & Multi-Tenancy (Days 11-13)

- Build the companies collection, register/login/OTP-verify routes
- Add JWT protection to every existing route
- Add `company_id` filtering to every database query — this is the most important security step in the whole project
- **Checkpoint:** two different company accounts cannot see each other's jobs or candidates, even if they guess an ID

PHASE 6 — React Dashboard (Days 14-18)

- Build the Auth pages (Login / Register / OTP) consuming your auth API
- Build the dashboard layout: sidebar navigation + top bar
- Build Jobs page (create + list), Candidates page (screen + list + detail), Pipeline page (run agents)
- **Checkpoint:** a brand new user can register, verify, log in, create a job, screen a resume, and run the full pipeline — entirely through the UI, zero Postman needed

PHASE 7 (STRETCH) — Agents 5, 6, 7 (Days 19+)

- Agent 5: extend the evaluator to score both text answers and submitted code
- Agent 6: integrate a speech-to-text/text-to-speech flow for the AI-led interview mode
- Agent 7: build simple scheduling logic — decide next round based on score thresholds, store a scheduled slot, send a notification email

TIME MANAGEMENT TIP

If you are on a tight deadline, Phases 0-6 alone (Agents 1-4 + full auth + full UI) make a complete, demoable, gradeable project. Treat Phase 7 as bonus marks, not a requirement — many successful student projects stop at Agent 4 and still score very well because what they built actually works end-to-end.

7. Database Design (MongoDB Collections)

HireFlow AI is **multi-tenant** — many companies share the same database, but each company's data is completely isolated. The mechanism for this is simple but you must apply it everywhere: **every document in every collection (except `companies` itself) carries a `company_id` field, and every single database query filters by it.**

```
companies
├── _id
├── email (unique)
├── password (bcrypt hash)
├── company_name
├── full_name
├── is_verified (bool)
└── created_at

jobs
├── _id
├── company_id        ← scopes this job to one company
├── job_title, required_skills, experience_required, ...
├── candidates_count
└── created_at

candidates
├── _id
├── company_id        ← scopes this candidate to one company
├── job_id            ← links to the job they applied for
├── candidate_name, match_score, matched_skills, ...
├── status            ← screened → shortlisted → invited → hired/rejected
├── interview_questions
└── created_at

otps
├── email
├── otp
└── expires_at        ← MongoDB TTL index auto-deletes expired OTPs
```

> **THE #1 SECURITY MISTAKE STUDENTS MAKE**
>
> Writing `jobs_collection.find_one({"_id": ObjectId(job_id)})` without also checking `company_id`. This means if Company A guesses or somehow obtains Company B's job ID, they can view Company B's private data. Every single read/write must look like: `find_one({"_id": ObjectId(job_id), "company_id": company_id})`. Make this a habit from the very first route you write.

Why MongoDB indexes matter here

An index lets MongoDB jump directly to matching documents instead of scanning the entire collection. Since you will constantly query "give me all jobs where `company_id` = X", you should create an index on `company_id` in both the `jobs` and `candidates` collections. Without this, performance degrades badly as your data grows — with it, lookups stay fast no matter how many companies sign up.

8. Deep Dive — Each of the 7 Agents

Every agent in this project follows the exact same skeleton. Learn this skeleton once and you can build all 7 agents by changing only the prompt and the validation rules.

```
def agent_function(input_data, context_data) -> dict:
    raw = ""                                  # always init before try
    try:
        prompt = PROMPT_TEMPLATE.format(...)  # fill in the template
        response = llm.invoke(prompt)          # call Gemini
        raw = response.content.strip()         # get text out
        raw = strip_markdown_fences(raw)        # clean ```json wrapper
        parsed = json.loads(raw)                # parse to dict
        validate_required_fields(parsed)        # sanity check
        return {"success": True, "data": parsed}
    except json.JSONDecodeError as e:
        return {"success": False, "error": str(e), "raw_response": raw}
    except Exception as e:
        return {"success": False, "error": str(e)}
```

Notice the contract: every agent returns `{"success": bool, "data"/"error": ...}`. This consistency means your Flask routes can handle any agent's result the exact same way, which makes the whole codebase predictable.

Agent 1 — JD Parser

Job: Convert unstructured job description text into structured JSON (title, skills, experience, responsibilities).

Temperature: 0.1 (low) — you want consistent, predictable extraction, not creative variation. The same JD given twice should produce nearly identical output.

Key design decision: the prompt explicitly tells Gemini to return ONLY JSON, with an exact schema shown inline in the prompt. This is the single most important prompting technique in this entire project — showing the model the exact shape you want, rather than describing it in prose, dramatically improves reliability.

Agent 2 — Resume Screener

Job: Compare a resume against a job's required skills and return a 0-100 match score with reasoning.

Temperature: 0.1 — scoring needs to be consistent. If you ran the same resume through twice, you want roughly the same score both times.

Key design decision: the prompt includes an explicit scoring rubric (e.g. "90-100 = exceptional match, 70-89 = strong match...") inside the prompt itself. Without this rubric, the LLM's notion of "what counts as an 80" drifts between different resumes — anchoring it with concrete bands keeps scores comparable across candidates.

EXAMPLE PROMPT SKELETON

"You are an expert technical recruiter. JOB REQUIREMENTS: {skills}... RESUME: {resume_text}... Return ONLY valid JSON with: match_score (0-100), matched_skills, missing_skills, recommendation. Scoring guide: 90-100 exceptional, 70-89 strong, 50-69 partial, below 50 weak."

Agent 3 — Interview Question Generator

Job: Generate 8 interview questions personalized to one specific candidate's profile — not generic questions.

Temperature: 0.7 (higher) — this is the one agent where you actually want creativity and variation. Two different candidates with the same missing skill should not get word-for-word identical questions.

Key design decision: the prompt feeds in the candidate's *matched skills*, *missing skills*, and *concerns* from Agent 2's output — not just the raw resume. This is "agent chaining": Agent 3 is smarter because it builds on Agent 2's analysis rather than starting from scratch. Questions are split into three categories — technical, behavioral, and gap-focused — so the output has clear structure an interviewer can actually use.

Agent 4 — Email Sender

Job: Write and send a warm, personalized shortlist notification email — under the hiring company's name, not HireFlow AI's name.

Temperature: 0.7 — natural, human-sounding writing benefits from some creative variation.

Key design decision (important!): this agent generates two things — the email *content* (via Gemini) and the actual email *delivery* (via Resend, a separate non-AI step). Keep these as two separate functions. The prompt explicitly instructs the model NOT to reveal interview questions in the email — real recruiters never send questions in advance, they send an invitation and confirm availability.

Agent 5 — Answer Evaluator

Job: Evaluate a candidate's interview answers — whether those answers came from a human-led interview, an AI-led interview (Agent 6's transcription), or a submitted coding exercise — and produce a final hiring recommendation.

Temperature: 0.2 (low-moderate) — evaluation needs to be fair and consistent across candidates, similar reasoning to Agent 2.

Key design decision: design this agent to accept an `answer_type` parameter (`"speech"` or `"code"`) and branch the prompt accordingly. For code answers, the prompt should ask the model to assess correctness, time/space complexity, and code style. For speech answers, it should assess clarity, depth, and technical accuracy. Both branches return the same output shape (score + reasoning + recommendation) so Agent 7 downstream doesn't need to know which type it was.

Agent 6 — AI Interviewer (Stretch Goal)

Job: When "AI Take" is selected, conduct the interview by speaking Agent 3's questions aloud and capturing the candidate's spoken answers as text.

This is not an LLM-reasoning agent — it's a pipeline of speech tools:

- **Text-to-Speech (TTS):** converts each question into audio the candidate hears. The simplest version for a student project is the browser's built-in `SpeechSynthesis` Web API (zero cost, zero backend work) — a more advanced version would use a cloud TTS API for more natural voices.
- **Speech-to-Text (STT):** captures the candidate's spoken response and converts it to text. The simplest version is the browser's `SpeechRecognition` Web API — works directly in Chrome with no extra backend cost.

Why this is correctly its own agent and not folded into Agent 3 or 5: Agent 3 decides *what to ask*. Agent 5 decides *how good the answer was*. Agent 6 is purely the *delivery mechanism* in between — converting text to speech and speech back to text. Mixing this responsibility into either neighbor would make both harder to test and maintain independently.

Agent 7 — Scheduler (Stretch Goal)

Job: Based on Agent 5's recommendation and the seniority of the role, decide what happens next — schedule an L2 technical round, schedule an HR round, or close out the candidate respectfully — then notify them.

This agent is mostly rule-based logic with a thin AI layer on top — for example: `if recommendation == "Strongly Recommend" and role_level == "senior": next_step = "L2 Technical"` . The AI's job here is small but real: writing the next-step notification email in natural language (similar pattern to Agent 4), and optionally helping phrase a polite rejection email when the candidate doesn't proceed.

For a college project, a "scheduled slot" can simply be a date/time field stored in MongoDB — you do not need a full calendar integration (like Google Calendar API) unless you want to attempt it as a true stretch goal.

9. Authentication & Multi-Tenancy Explained

This is the part students most often underestimate, but it's what turns your project from "a script that works" into "a real product." Walk through the flow carefully:

```
1. REGISTER
   User submits: full_name, email, password, company_name
   → Backend hashes password with bcrypt (never store plain text!)
   → Backend creates company doc with is_verified = False
   → Backend generates a 6-digit OTP, stores it with a 1-min expiry
   → Backend emails the OTP via Resend

2. VERIFY OTP
   User submits: email, otp
   → Backend checks OTP matches AND hasn't expired
   → Backend sets is_verified = True
   → Backend issues a JWT containing { company_id, email, company_name }

3. LOGIN (returning users)
   User submits: email, password
   → Backend checks bcrypt hash matches
   → Backend issues a fresh JWT (valid 8 hours in this project)

4. EVERY SUBSEQUENT REQUEST
   Frontend sends: Authorization: Bearer
   → Backend decodes JWT, extracts company_id
   → Backend filters EVERY database query by that company_id
```

WHY JWT INSTEAD OF JUST A SESSION ID?

A JWT is self-contained — it carries the company's identity inside the signed token itself, so the backend doesn't need to look anything up in a database to know who's making the request. This makes the backend "stateless," which is a key property for scaling a system later (you could run 10 copies of your Flask backend and any of them could validate any token, with no shared session storage needed).

A USEFUL MENTAL TEST FOR MULTI-TENANCY

Before writing any database query, ask yourself: "If I were Company B, could I somehow see Company A's data through this exact query?" If the answer is yes, you're missing a `company_id` filter. Run this test on every single route you write — it becomes second nature after a few routes.

10. AI Prompt Engineering Recommendations

These are the techniques that made all 7 agents in this guide reliable. Apply them consistently and your agents will behave predictably; skip them and you'll get inconsistent, hard-to-debug JSON parsing failures.

1. Always show the exact JSON shape you want

WEAK PROMPT

"Extract the skills and experience from this job description."

STRONG PROMPT

"Return ONLY a valid JSON object with exactly this structure: {"job_title": "...", "required_skills": ["...", "..."], "experience_required": "..."}. No markdown, no explanation, return ONLY the JSON."

2. Match temperature to the task

Use **0.0 - 0.2** for anything that needs consistency — parsing, scoring, evaluation. Use **0.6 - 0.8** for anything that benefits from variety — question generation, email writing. Never use high temperature for tasks where you compare outputs across candidates (like scoring) — inconsistent scoring defeats the purpose of automation.

3. Always strip markdown fences before parsing JSON

LLMs frequently wrap JSON responses in ```` ```json ... ``` ```` even when told not to. Always run a regex strip before calling `json.loads()`:

```
raw = re.sub(r"```json\s*|\s*```", "", raw).strip()
```

4. Validate critical fields after parsing

Don't trust that the JSON has every field you expect just because it parsed successfully. After `json.loads()`, explicitly check that fields like `required_skills` or `match_score` exist and raise a clear error if not — this turns a silent bug into a debuggable one.

5. Give the model a scoring rubric inline, not just a 0-100 range

"Return a score 0-100" alone produces wildly inconsistent results. "90-100 = exceptional, 70-89 = strong, 50-69 = partial, below 50 = weak" anchors the model's judgment and produces far more comparable scores across different resumes.

6. Pass downstream agents the upstream agent's reasoning, not just raw data

Agent 3 should receive Agent 2's `matched_skills`, `missing_skills`, and `concerns` — not just the raw resume text. This is what makes the pipeline feel intelligent rather than having each agent reinvent analysis from scratch.

11. Common Errors & How to Debug Them

Every team building this project hits some version of these errors. Recognizing the pattern saves you hours.

Error message	Real cause	Fix
`ModuleNotFoundError: No module named 'X'`	Package not installed, or you're running the wrong Python (global instead of venv)	Check your terminal shows `(venv)` before running. Re-run `pip install X`
`cannot import name 'PromptTemplate' from 'langchain_core'`	Wrong import path	Use `from langchain_core.prompts import PromptTemplate`, never import it from the top-level package
`UnboundLocalError: cannot access local variable 'raw'`	Variable referenced in an except block before being assigned in the try block	Always initialize `raw = ""` before the try block starts
`can't compare offset-naive and offset-aware datetimes`	Mixing `datetime.utcnow()` (naive) with MongoDB's timezone-aware datetimes	Always use `datetime.now(timezone.utc)` consistently everywhere
`pymongo.errors.InvalidURI: ... must be escaped`	Special characters (like `@` or `!`) in your MongoDB password	URL-encode the password using `urllib.parse.quote_plus()` before putting it in your `.env`
`429 RESOURCE_EXHAUSTED` from Gemini	Free tier rate limit hit — too many requests too fast	Wait 30-60 seconds between test calls during development, or switch to a lighter model variant
`403 PERMISSION_DENIED ... suspended`	API key flagged for unusual usage	Generate a fresh API key from your AI provider's console
`TypeError: 'bool' object is not callable`	Calling a Flask property as if it were a function, e.g. `request.is_json()`	Properties don't use parentheses — use `request.is_json` without `()`
`AssertionError: View function mapping is overwriting an existing endpoint`	Two routes in the same blueprint have ambiguous or duplicate URL patterns	Give each route a distinct, unambiguous URL path, e.g. `/job/<job_id>` vs `/<candidate_id>`
`E11000 duplicate key error`	Trying to insert a document with a value that already exists in a uniquely-indexed field	Delete the old conflicting record first, or check for existing records before inserting

> **GENERAL DEBUGGING HABIT**
> When Flask shows a traceback, read it from the **bottom up** — the last line is almost always the actual error message; everything above it is just the chain of function calls that led there. Find the file and line number Python points to, open that exact line, and read it literally before assuming anything complex is wrong.

12. Evaluation Checklist — Is My Project Complete?

Use this as a self-assessment before submission. Each item should be something you can actually demonstrate live, not just claim.

☐ Backend runs without errors; `/api/health` returns a 200 response

☐ A new company can register, receive an OTP email, verify it, and log in

☐ An unverified or logged-out user cannot access protected routes (test this — try hitting an API route without a token)

☐ Two different company accounts cannot see each other's jobs or candidates

☐ Agent 1 correctly extracts structured fields from at least 3 different JD formats (text, PDF, Word)

☐ Agent 2 produces a sensible match score with clear matched/missing skills for a real resume

☐ Agent 3 generates genuinely different questions for two candidates with different skill gaps

☐ Agent 4 successfully delivers a real email to a real inbox (check spam folder if missing)

☐ All error cases return clean, human-readable JSON error messages — not raw Python tracebacks

☐ The React dashboard lets a user complete the entire flow (create job → screen resume → generate questions → send email) without ever opening Postman

☐ The UI clearly shows a candidate's status as it moves through the pipeline (screened → shortlisted → invited)

☐ You can explain, out loud, why each technology in the stack was chosen (refer back to Section 5)

13. Stretch Goals (For Extra Marks)

If your core pipeline (Agents 1-4 + auth + dashboard) is solid and you have time remaining, these additions demonstrate deeper engineering maturity:

- **Build Agent 5 (Answer Evaluator)** — start with text-based answers only, add code evaluation as a follow-up
- **Build a simple version of Agent 6** — use the browser's free Web Speech API for both text-to-speech and speech-to-text; no paid service required to demonstrate the concept
- **Build Agent 7's scheduling logic** — even a simple rule-based decision (score above X → L2, below X → reject) stored in MongoDB counts as a real implementation
- **Add a candidate-facing portal** — let candidates view their own application status via a unique link, without needing a full login
- **Add resume bulk upload** — let a recruiter upload 10 resumes at once and screen them all in one batch
- **Add analytics** — a chart showing average match scores per job, candidates per pipeline stage, time-to-hire trends
- **Containerize with Docker** — package the Flask backend, React frontend, and environment config so the whole project can be started with one command on any machine